DeutschLingoDeck – Use Cases

1. Authentication Use Cases

UC-001 – Register a New User
The user can create a new account by providing an email address, password, and display name. The Angular application sends a request to `POST /auth/register`. Upon successful registration, the backend returns an access token, a refresh token, and the authenticated user's basic information.

UC-002 – User Login
The user can authenticate using an email address and password. The Angular application sends a request to `POST /auth/login`. After a successful login, the tokens are securely stored on the client side and the user is redirected to the application dashboard.

UC-003 – Refresh Access Token
When the access token expires, the Angular application automatically sends the refresh token to `POST /auth/refresh` in order to obtain a new access token without requiring the user to log in again.

UC-004 – Logout
The user can log out of the application. Angular sends a request to `POST /auth/logout`, removes all locally stored authentication tokens, and redirects the user to the login page.

UC-005 – Load Current User
After the application starts or the page is refreshed, Angular calls `GET /auth/me` to retrieve information about the currently authenticated user and restore the user session.

---

2. Profile Use Cases

UC-006 – View User Profile
The user can open the profile page. Angular sends a request to `GET /profile` and displays profile information such as email address, display name, and account creation date.

UC-007 – Update Profile
The user can update profile information, such as the display name. Angular submits the updated data to `PUT /profile`.

UC-008 – Change Password
The user can change the account password by providing the current password and a new password. Angular sends the request to `PUT /profile/password`.

---

3. Dictionary Use Cases

UC-009 – View Dictionary List
The user can view a paginated list of all personal dictionaries. Angular retrieves the data using `GET /dictionaries` with pagination parameters.

UC-010 – Import Dictionary from Excel
The user can upload a custom vocabulary dictionary in Microsoft Excel format. Angular sends a `multipart/form-data` request to `POST /dictionaries/import`, including the file, dictionary name, description, source language, and target language.

UC-011 – View Dictionary Details
The user can open a specific dictionary and view its metadata. Angular retrieves the information using `GET /dictionaries/{dictionaryId}`.

UC-012 – Update Dictionary Metadata
The user can edit dictionary information, including its name, description, source language, and target language. Angular sends the updated data using `PUT /dictionaries/{dictionaryId}`.

UC-013 – Delete Dictionary
The user can remove a dictionary from the active collection. Angular sends `DELETE /dictionaries/{dictionaryId}`. The backend performs a soft delete to preserve historical game data.

UC-014 – View Dictionary Cards
The user can browse all vocabulary cards belonging to a selected dictionary. Angular requests the data from `GET /dictionaries/{dictionaryId}/cards` with pagination support.

UC-015 – View Card Details
The user can inspect detailed information about a specific vocabulary card. Angular retrieves the data using `GET /dictionaries/{dictionaryId}/cards/{cardId}`.

---

4. Game Use Cases

UC-016 – Start a New Game
The user selects a dictionary and starts a new learning session. Angular sends a request to `POST /games`, including the selected dictionary identifier and optionally the desired game mode.

UC-017 – Retrieve Current Game State
The user can continue an interrupted learning session after refreshing the browser or reopening the application. Angular calls `GET /games/{gameId}` to retrieve the current game state.

UC-018 – Submit an Answer
The user submits an answer for the currently displayed card. Angular sends the answer, card identifier, and response time to `POST /games/{gameId}/answers`.

UC-019 – Receive Answer Validation
After each submitted answer, the backend validates the response and returns detailed feedback. Possible validation results include:

- CORRECT
- WRONG_ARTICLE
- WRONG_TRANSLATION
- TYPO
- MISSING_WORD
- EXTRA_WORD
- PARTIALLY_CORRECT
- WRONG_SENTENCE

Angular displays appropriate feedback to the user.

UC-020 – Continue to the Next Card
If another card is available, the backend returns it as part of the response. Angular immediately displays the next vocabulary card without reloading the page.

UC-021 – Finish a Game
The user completes the learning session. Angular sends `POST /games/{gameId}/finish` and displays the final game summary.

UC-022 – Abandon a Game
The user may terminate the current learning session before completion. Angular sends `POST /games/{gameId}/abandon`. All previously submitted answers remain stored.

UC-023 – View Game Summary
The user can review the final statistics of a completed game. Angular retrieves the summary using `GET /games/{gameId}/summary`.

UC-024 – View Game History
The user can browse previous learning sessions. Angular retrieves paginated results using `GET /games`, optionally filtering by dictionary or game status.

---

5. Statistics Use Cases

UC-025 – View Dashboard Statistics
The user can access the dashboard displaying overall learning statistics. Angular retrieves the information using `GET /statistics`, optionally filtered by date range.

UC-026 – View Card Statistics
The user can analyze statistics for individual vocabulary cards. Angular requests the data using `GET /statistics/cards`, optionally filtered by dictionary.

UC-027 – View Dictionary Statistics
The user can review learning statistics grouped by dictionary. Angular retrieves the information using `GET /statistics/dictionaries`.

UC-028 – View Learning History
The user can explore the complete learning history, including previous games and performance over time. Angular retrieves the data using `GET /statistics/history` with optional date filters and pagination.

---

6. Cross-Cutting Frontend Use Cases

UC-029 – Display Validation Errors
The application displays field-level validation errors returned by the backend, such as invalid email addresses, missing required fields, invalid passwords, or incorrect file uploads.

UC-030 – Display Business Errors
The application displays business-level errors returned by the backend, such as:

- Dictionary already exists
- Game has already been completed
- Resource not found
- Import validation failed

UC-031 – Attach JWT Token Automatically
Every authenticated API request automatically includes the JWT access token using an Angular HTTP interceptor.

UC-032 – Automatic Token Refresh
When the backend returns an HTTP 401 Unauthorized response due to an expired access token, Angular automatically refreshes the token and retries the original request without interrupting the user's workflow.

UC-033 – Protect Secure Routes
Angular Route Guards prevent unauthenticated users from accessing protected areas of the application, including:

- Dashboard
- Dictionaries
- Games
- Statistics
- Profile

Unauthenticated users are automatically redirected to the login page.

UC-034 – Handle Application States
Every page consistently supports the following UI states:

- Loading state while data is being retrieved.
- Success state when data is successfully loaded.
- Empty state when no data is available.
- Error state when an unexpected error occurs.

This ensures a consistent and user-friendly experience across the entire application.

UC-035 – Resume Active Game
The user may leave the application during an active learning session by refreshing the browser, closing the tab, or temporarily leaving the application. When the user returns, the Angular application automatically checks whether there is an unfinished game available by retrieving the current game state from the backend. If an active game exists, the user is offered the option to resume the previous session instead of starting a new one. If no active game exists, the application behaves normally and allows the user to start a new game.

UC-036 – Display Dictionary Import Validation Report
When importing a vocabulary dictionary from an Excel file, the backend may detect warnings or validation issues such as duplicated entries, empty rows, unsupported formats, or partially imported records. After the import process completes, Angular displays a detailed validation report summarizing the number of successfully imported cards, skipped records, warnings, and validation messages. The user can review the report before continuing to use the imported dictionary.

UC-037 – Confirm Dictionary Deletion
Before deleting a dictionary, Angular displays a confirmation dialog explaining that the dictionary will no longer be available for future learning sessions. The dialog clearly informs the user that historical learning statistics and completed game history will remain preserved

because the backend performs a soft delete. The dictionary is deleted only after the user explicitly confirms the action.

UC-038 – Redirect Authenticated User Away from Login and Registration Pages
If an already authenticated user attempts to access the Login or Registration pages, Angular automatically redirects the user to the application dashboard instead of displaying the authentication forms. This prevents authenticated users from unnecessarily accessing authentication pages and improves the overall user experience.

UC-039 – Handle Forbidden Access
If the backend responds with an HTTP 403 Forbidden status because the authenticated user attempts to access a resource that does not belong to them or for which they do not have sufficient permissions, Angular displays a dedicated access denied page or an appropriate notification explaining that the requested operation is not permitted. The application remains fully functional and allows the user to continue using authorized features.

UC-040 – Global Application Navigation
The application provides a consistent navigation experience through a global navigation layout that is available on every authenticated page. The navigation allows users to access the Dashboard, Dictionaries, Games, Statistics, Profile, and Logout actions. The currently active page is visually highlighted, and navigation remains responsive across desktop and mobile devices. The navigation layout also displays the currently authenticated user's display name and provides quick access to profile settings and logout functionality.

UC-041 – Handle Expired Session
If both the access token and refresh token have expired or become invalid, Angular automatically clears all locally stored authentication data, redirects the user to the Login page, and displays a notification explaining that the session has expired and a new login is required.

UC-042 – Display Empty States
Whenever a page has no available data (for example, no dictionaries, no completed games, or no learning statistics), Angular displays a meaningful empty state with an explanation and a clear call-to-action guiding the user toward the next logical step, such as importing a dictionary or starting the first learning session.

UC-043 – Preserve User Context After Page Refresh
After a browser refresh, Angular restores the user's authentication state, reloads the current user information, and reconstructs the current application state whenever possible, minimizing disruption to the user's workflow.

UC-044 – Global Error Handling
Unexpected server errors, network failures, or unavailable backend services are handled by a centralized Angular error handling mechanism. The application displays user-friendly error messages while logging technical details for debugging purposes and prevents the application from entering an inconsistent state.